PROJECT REQUIREMENTS & TECHNICAL SPECIFICATION

StudyConnect API

The Peer-to-Peer Student Tutoring & Academic Platform

DOCUMENT METADATA

Document Version: 1.0 (Production & Defense Edition)

Architecture: NestJS (TypeScript) + TypeORM + PostgreSQL + Redis

Target Cloud Hosting: Render.com Web & Database Service

Notification System: Resend Email API (OTP Verification)

Project Type: Peer-to-Peer Tutoring & Academic Resource Hub

Date: October 2026

Comprehensive blueprint for development, quality assurance, deployment, and academic evaluation.

1. Executive Summary & Problem Statement

University students frequently encounter academic challenges with specific demanding courses or topics (such as Calculus in MTH 102 or Data Structures in CSC 201). Simultaneously, academically strong peer students are willing to help but lack a safe, organized campus medium to connect, schedule tutorials, and share learning materials.

StudyConnect resolves this by enabling a dual-role peer-to-peer ecosystem. Students do not choose to be solely learners or tutors; they can tutor courses where they excel while receiving peer academic assistance in courses they find challenging.

2. The 5-Layer Backend Architecture

The system is built on a clean 5-layer separation of concerns in NestJS, ensuring zero misplaced code:

• **Layer 1 - Client Request:** Frontend clients send HTTP requests with JSON bodies and Bearer JWT authorization headers.

• **Layer 2 - Controller Layer:** Listens to HTTP routes, validates input with class-validator DTOs, and enforces JWT guards.

• **Layer 3 - Service Layer:** Executes all core business logic, hashes passwords with bcryptjs, and handles rating algorithms.

• **Layer 4 - Repository Layer:** Abstracts data queries and encapsulates TypeORM operations from business logic.

• **Layer 5 - Entity & PostgreSQL:** TypeORM TypeScript entities map strictly to PostgreSQL relational tables, ensuring 100% ACID consistency.

3. Technology Choices & Justification

Every technology in this stack was selected for reliability, team collaboration speed, and academic defensibility:

• **NestJS over Plain Express:** Enforces strict modular folders (controllers, services, entities, DTOs), preventing displaced code in multi-developer teams.

• **PostgreSQL over MongoDB:** Students, bookings, courses, and ratings are strictly relational. Foreign keys eliminate corrupted or orphaned records.

• **TypeORM over Raw SQL:** Translates TypeScript entity classes directly into database schemas with automatic validation and compile-time type safety.

• **Redis Cache & Throttler:** Stores 10-minute temporary OTP codes and rate-limiting counters in memory for sub-millisecond response times without burdening the main database.

• **Resend Email API:** Reliable transactional email service delivering 6-digit verification OTPs to students without server maintenance overhead.

• **Bcryptjs:** Pure JavaScript implementation of salt-based password hashing that runs cleanly across all operating systems without native build failures.

• **Render.com:** Provides unified cloud hosting for PostgreSQL and Node.js with automated continuous deployment on git push.

4. Detailed Feature Specifications & Technical Realization

4.1 Student Authentication & OTP Email Verification

• **Functional Flow:** Student registers with university email, department, level, and picks role (TUTOR, LEARNER, or BOTH). An unverified account is created and a 6-digit OTP code (valid for 10 minutes) is sent via Resend. A 60-second cooldown prevents spam.

• **Technical Aspect:** POST /api/v1/auth/register hashes password with bcryptjs and stores user as unverified. POST /api/v1/auth/verify-otp checks OTP in Redis, verifies account, and returns JWT. POST /api/v1/auth/resend-otp is rate-limited via NestJS Throttler.

4.2 Student Profiles & Academic Course Preferences

• **Functional Flow:** Students specify course codes they can teach (e.g. CSC201, MTH101) and courses where they need help (e.g. PHY102), along with a bio and contact info.

• **Technical Aspect:** Managed via GET /api/v1/users/profile/me and PATCH /api/v1/users/profile/me with TypeORM array/relation mapping.

4.3 Course Catalog & Topic-Based Granular Search

• **Functional Flow:** A student searching "Integration" or "MTH 102" finds matching curriculum topics, registered tutors, and live demand counts.

• **Technical Aspect:** GET /api/v1/courses/search?q=Integration executes indexed case-insensitive ILIKE matching on courses and topics with standard relational SQL counting.

4.4 1-on-1 and Group Tutoring Sessions & Scheduling

• **Functional Flow:** Students book sessions by picking date, time, duration, session type (1-on-1 or group max 5), and a physical meetup location or Google Meet link.

• **Technical Aspect:** POST /api/v1/tutoring/sessions creates a session with SCHEDULED status. Tutors and learners view schedules via GET /api/v1/tutoring/my-sessions.

4.5 Study & Reading Partner Matching with Reshuffle

• **Functional Flow:** Classmates taking the same courses are paired as reading partners. If a partner is inactive, clicking "Reshuffle" finds a new peer.

• **Technical Aspect:** POST /api/v1/partners/match pairs complementary learners; POST /api/v1/partners/:id/reshuffle marks current pair RESHUFFLED and queries the next candidate.

4.6 Attendance Tracking & Verification

• **Functional Flow:** When a tutorial ends, the tutor marks attendance. The learner's attended sessions count increments and the session is marked completed.

• **Technical Aspect:** POST /api/v1/attendance/:id/verify updates attended status in session_participants and increments stats in a database transaction.

4.7 Tutor Ratings, Reviews & Testimonials

• **Functional Flow:** Learners rate verified tutors from 1 to 5 stars and leave written testimonials. Tutor average rating updates automatically.

• **Technical Aspect:** POST /api/v1/reviews validates that attendance was completed, then re-aggregates the tutor average rating in PostgreSQL.

4.8 Academic Material Sharing Hub

• **Functional Flow:** Students upload and download past examination papers, lecture notes, and cheat-sheets categorized by course code. Max size 10MB.

• **Technical Aspect:** POST /api/v1/materials/upload checks MIME types (strictly PDF, DOCX, PNG, JPG) and stores files safely with download counters.

4.9 Student Safety & Conduct Reporting

• **Functional Flow:** Students can report inappropriate conduct or missed meetups with one click for administrative review.

• **Technical Aspect:** POST /api/v1/reports creates moderation records with reporter ID, reported ID, reason, and status flags.

5. Database Schema & Data Dictionary

All tables inherit UUID primary keys and timestamp tracking from BaseEntity:

- **users:** id, email, password, role, department, level, coursesToTeach, coursesNeedingHelp, rating, isVerified
- **courses:** id, code (e.g. MTH102), title, department, level
- **topics:** id, courseId (FK), title (e.g. Integration), difficulty
- **tutoring_sessions:** id, tutorId (FK), courseId (FK), sessionType, locationVenue, meetingUrl, scheduledAt, status
- **session_participants:** id, sessionId (FK), learnerId (FK), attended (Boolean)
- **study_partner_matches:** id, studentAId (FK), studentBId (FK), courseId (FK), status (ACTIVE/RESHUFFLED)
- **materials:** id, uploaderId (FK), courseId (FK), title, fileUrl, fileType, downloadsCount
- **reviews:** id, sessionId (FK), tutorId (FK), learnerId (FK), rating (1-5), testimonial
- **reports:** id, reporterId (FK), reportedUserId (FK), reason, description, status (PENDING/RESOLVED)

6. Security & Standardized Envelope Architecture

- **Response Envelope:** All successful responses are structured as { success: true, statusCode, message, data, timestamp }.
- **Exception Filter:** All errors are returned uniformly as { success: false, statusCode, message, path, timestamp }.
- **Rate Limiting:** OTP requests limited to 1 per 60s per IP; login attempts capped at 5 per 15 minutes to prevent brute-force attacks.

7. Cloud Deployment Guide (Render.com)

- **Step 1: Managed PostgreSQL:** Create a free PostgreSQL instance on Render named study-connect-db and copy the connection credentials.
- **Step 2: Web Service:** Connect the GitHub repository to Render as a Node Web Service using build command: npm install && npm run build.
- **Step 3: Environment Configuration:** Set PORT=3000, NODE_ENV=production, database credentials, JWT_SECRET, and RESEND_API_KEY.
- **Step 4: Live Verification:** Verify health at /api/v1/health and interact with the live Swagger playground at /docs.